Shani “Dill” De Silva, U44033199

EC 551 Advanced Digital Design with Verilog and FPGAs

Fall 2026, Prof. Douglas Densmore

Homework #2

Due: 10/20

1. Find a comparison chart of FPGAs (e.g., within product families is fine; Altera is an older FPGA maker acquired by Intel in 2015 that you might want to use). Highlight four differences between devices and discuss how these might be either beneficial for a design or detrimental to a design. Discuss these in terms of performance, power, and cost. In addition, briefly discuss the metrics by which the FPGAs are compared (e.g. the columns in the chart perhaps).

I have chosen to compare the Agilex® 5 FPGA E-Series 005B (B15A) and the Agilex® 5 FPGA E-Series 065A (B32A), which are in the same series but different enough that they should be interesting to compare.

Difference in LUTs per FPGA: the 005B has 17100 and the 065A has 222400. Both chips have the same ratio of LUTs to logic registers, but the 065A is a far larger FPGA which is helpful for larger designs.

Difference in Maximum Embedded Memory: the 005B has 3.06Mb on-chip, and the 065A has 38.25Mb available. The difference in memory correlates to a difference in power and cost.

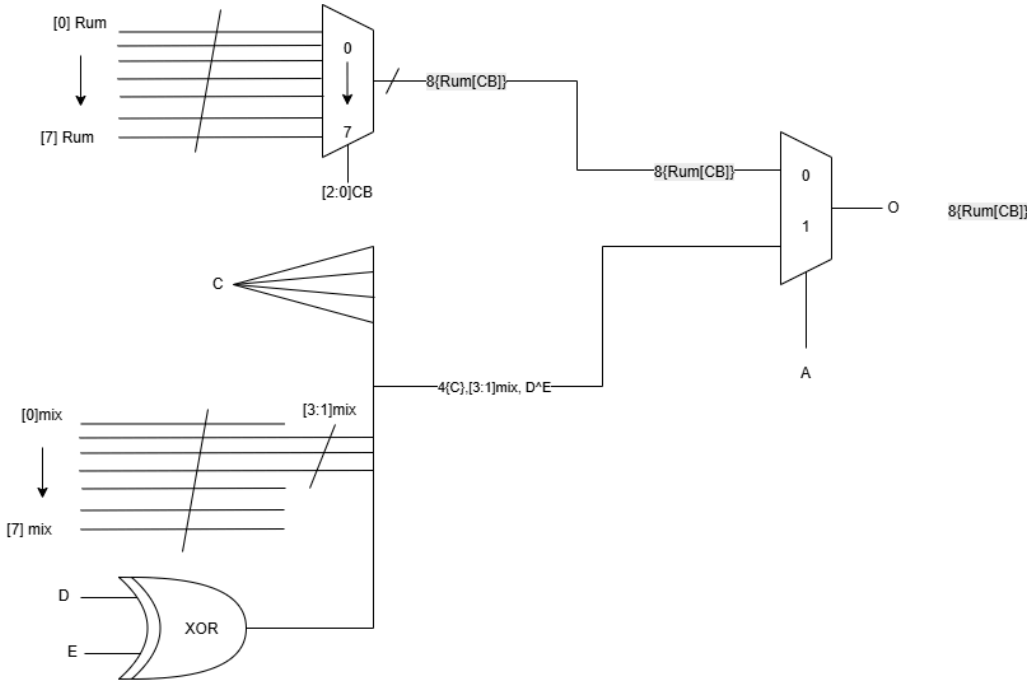
Difference in Fabric Speed Grades: the 005B has a speed grade of 4, 5, 6 while the 065A has a speed grade of 1, 2, 3. Smaller numbers are faster - the 065A has a higher performance. This is useful, especially when coupled with the fact that the 065A will likely be used to run larger synthesised code.

Difference in Peak INT 8 (TOPS): This refers to the theoretical maximum number of 8 bit operations per second, divided by 1 trillion. The 005B has 1.7, and the 065A has 25.99, which is an incredibly large difference in performance.

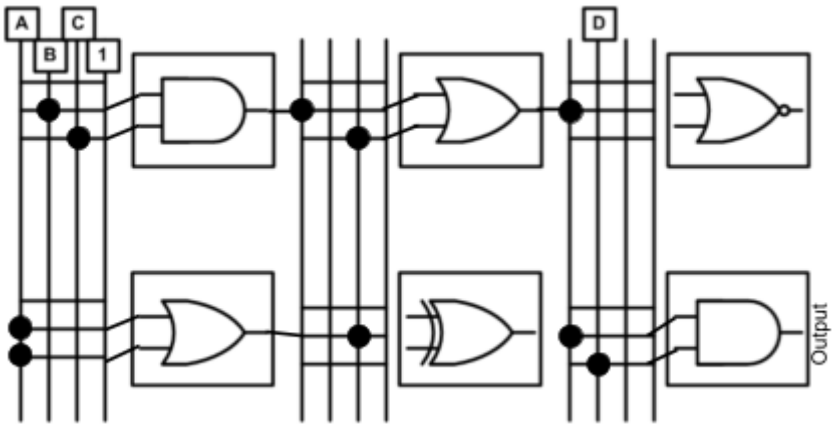
The metrics provided seemed to mainly affect size/area (changing costs) and performance, which is probably to be expected as I chose to compare two FPGAs from the same company and same release date. I didn’t find much information on power usage.

2. Draw a schematic showing what the following Verilog code is describing.

```
reg [7:0] O, mix, Rum;
reg [2:0] CB;
reg A, C, D, E;
assign O = A? {{4{C}}, mix[3:1], D^E} : {8{Rum[CB]}};
```



3. a. Connect (draw lines from gate inputs and outputs and/or add dark circles on the switch fabric) the primary inputs, gate inputs, and gate outputs to the provided FPGA switch fabric to realize the function: **(A+BC)D**



b. Assuming this design has only six 2-input LUTs and the six illustrated 4x4 crossbars, how many bits of SRAM would be needed to store connection and LUT information? (show work, state assumptions)

6 LUTS * 3 pins per LUT * 4 possible connections per pin = 72 bits
6 LUTS * 2 inputs ^2 = 24 bits (LUT outputs)
72+24=96 bits

I am assuming that a LUT cannot pass information backwards in this diagram.

c. How realistic is this FPGA? What is it missing (if anything)? How would you make this better? What would be some challenges of making this FPGA?

Not very realistic - quite small! Also, the LUTs are pre-defined as gates, instead of allowing me to program them myself. A way to improve this setup would be to add vertical wires, to make it easier to pass A to the next layer of LUTs. A challenge of making this FPGA would be that the gates come pre-decided, so it isn’t really an FPGA?

4. Create a circuit that counts the number of 1’s in a 4-bit value. The input should be 4-bits wide and the output should be a 3-bit binary number representing the value.

Provide verilog code, schematics, and relevant synthesis messages for all four parts.

a. Write a structural Verilog description of the design that synthesizes.

```
module four_a(
    input  [3:0] four_in,
    output wire [2:0] three_out
);
wire and_01, and_23; //for the [2] digit
and(and_01, four_in[0], four_in[1]);
and(and_23, four_in[2], four_in[3]);
and(three_out[2], and_01, and_23);


wire xor_01, xor_23; //for the [0] digit
xor(xor_01, four_in[0], four_in[1]);
xor(xor_23, four_in[2], four_in[3]);
xor(three_out[0], xor_01, xor_23);


//wire and_01, and_23 reused from calculation for [2] digit.
wire and_02, and_03, and_12, and_13;
```

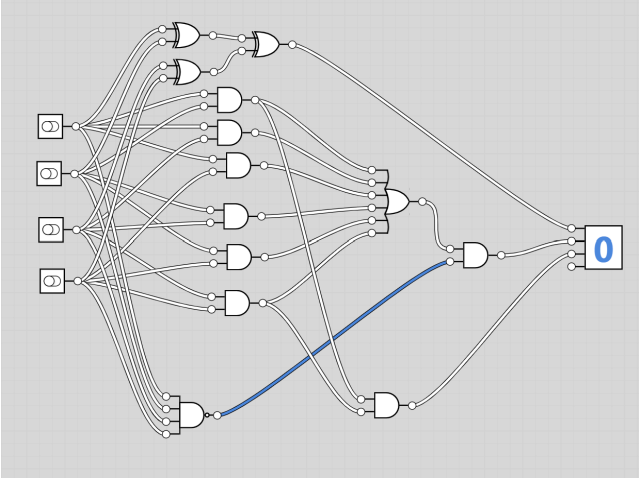
```
wire or_allpairs, nand_0123;
//and(and_01, four_in[0], four_in[1]); already calculated
and(and_02, four_in[0], four_in[2]);
and(and_03, four_in[0], four_in[3]);
and(and_12, four_in[1], four_in[2]);
and(and_13, four_in[1], four_in[3]);
//and(and_23, four_in[2], four_in[3]); already calculated

or(or_allpairs, and_01, and_02, and_03, and_12, and_13, and_23);
nand(nand_0123, four_in[0], four_in[1],four_in[2], four_in[3]);
and(three_out[1], or_allpairs, nand_0123);

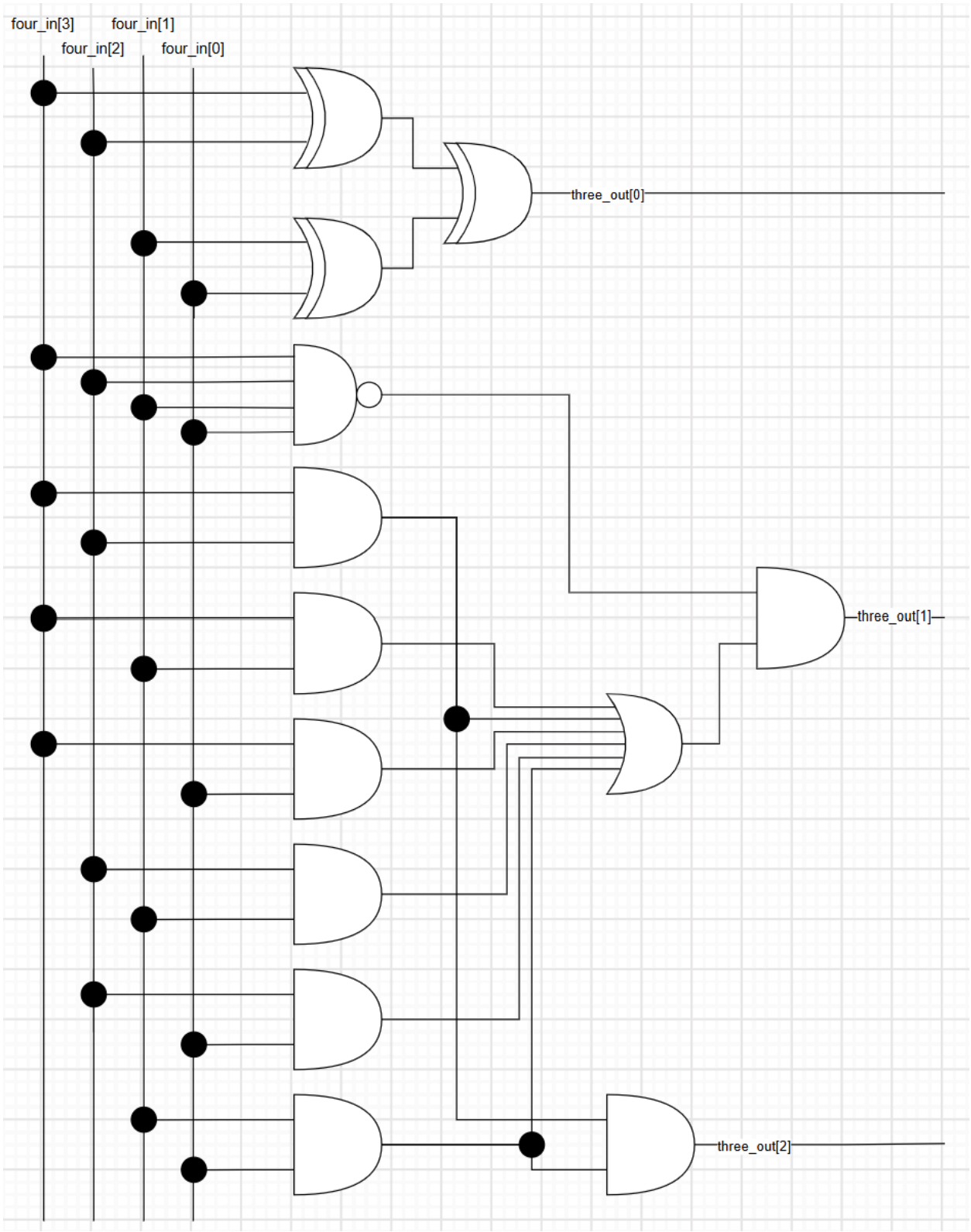
endmodule


module test;
    reg  [3:0] in_test;
    wire [2:0] out_test;
    countones dut(in_test, out_test);
    initial begin
        in_test = 4'b0000; #5 $display("in=0000 -> out=%h%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0001; #5 $display("in=0001 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0011; #5 $display("in=0011 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0111; #5 $display("in=0111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b1111; #5 $display("in=1111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
    end
endmodule
```

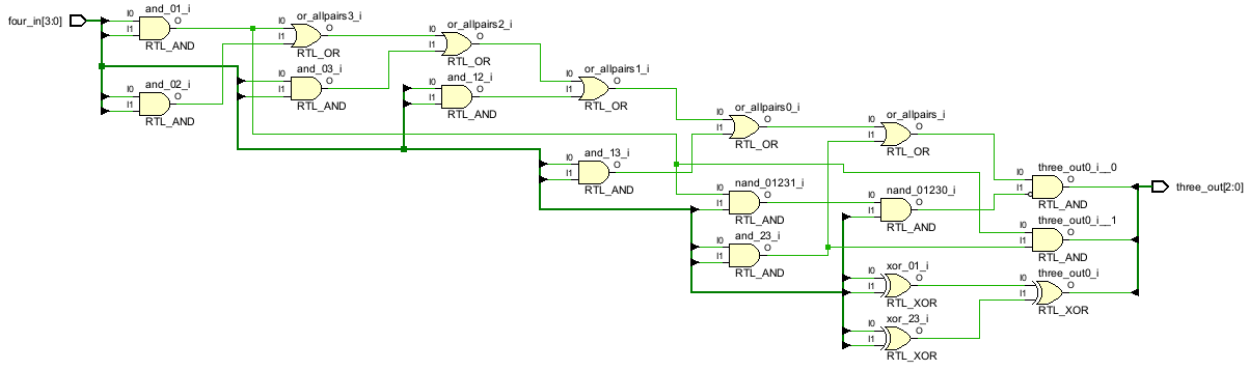
Used logicly demo (https://logic.ly/demo/) to test out logic first:



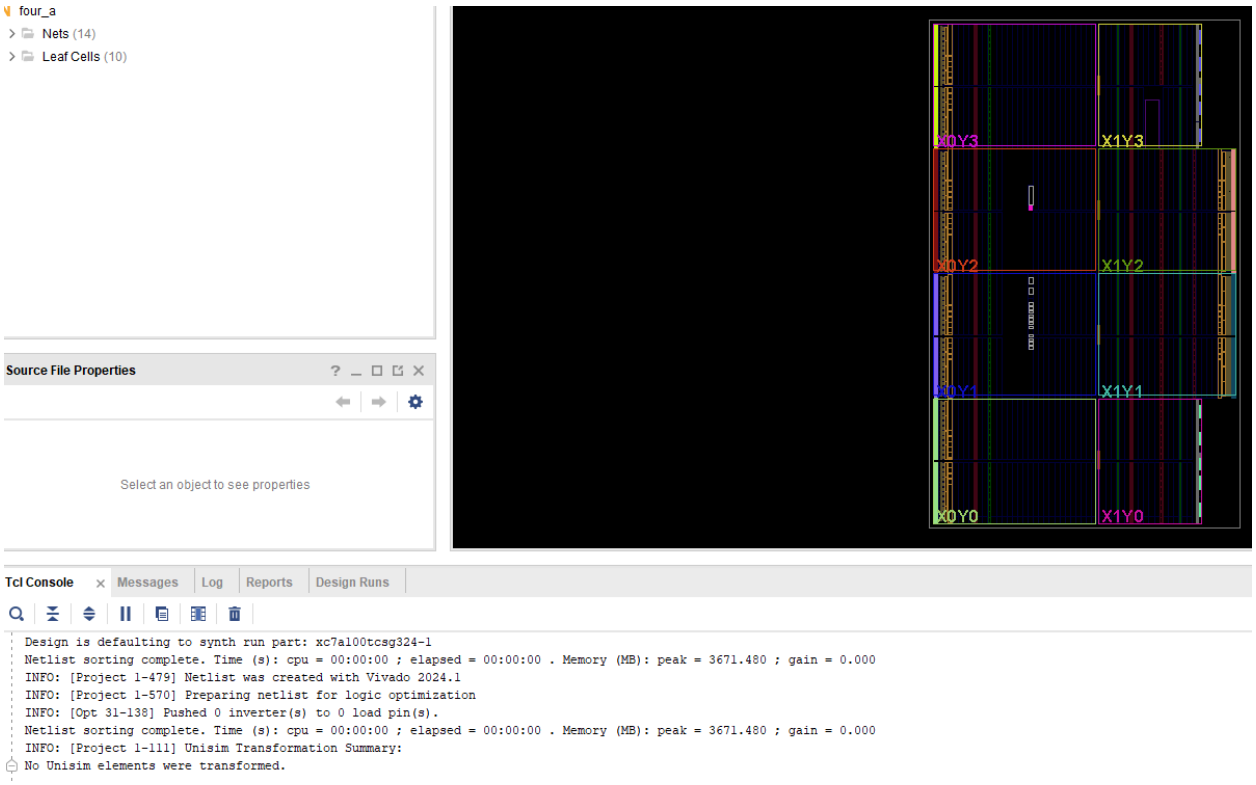
Leading to the logic schematic below, drawn on draw.io:



LOGIC FROM VIVADO:




```
Starting static elaboration
Pass Through NonSizing Optimizer
Completed static elaboration
INFO: [XSIM 43-4323] No Change in HDL. Linking previously generated obj files to create kernel
INFO: [USF-XSim-69] 'elaborate' step finished in '1' seconds
INFO: [USF-XSim-4] XSim::Simulate design
INFO: [USF-XSim-61] Executing 'SIMULATE' step in 'C:/Users/sdsva/Documents/GitHub/bu-EC551/hw/hw2/verilogtesting/project_1/project_1.sim/sim_1/behav/xsim'
INFO: [USF-XSim-98] *** Running xsim
   with args "four_a_dut_behav -key {Behavioral:sim_1:Functional:four_a_dut} -tclbatch {four_a_dut.tcl} -log {simulate.log}"
INFO: [USF-XSim-8] Loading simulator feature
Time resolution is 1 ps
source four_a_dut.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File-
#   }
# }
# run 1000ns
in=0000 -> out=000
in=0001 -> out=001
in=0011 -> out=010
in=0111 -> out=011
in=1111 -> out=100
INFO: [USF-XSim-96] XSim completed. Design snapshot 'four_a_dut_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```



open_run impl_1
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 3671.480 ; gain = 0.000
INFO: [Project 1-479] Netlist was created with Vivado 2024.1
INFO: [Project 1-570] Preparing netlist for logic optimization
Read ShapeDB Complete: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.001 . Memory (MB): peak = 3671.480 ; gain = 0.000
INFO: [Designutils 20-5722] Start Reading Physical Databases.
Reading placement.
Read Netlist Cache: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 3671.480 ; gain = 0.000
Reading placer database...
Read Device Cache: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.006 . Memory (MB): peak = 3671.480 ; gain = 0.000
Read PlaceDB: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 3671.480 ; gain = 0.000
Read PulsedLatchDB: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 3671.480 ; gain = 0.000
Reading routing.
Read RouteStorage: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.026 . Memory (MB): peak = 3671.480 ; gain = 0.000
Read Physdb Files: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.038 . Memory (MB): peak = 3671.480 ; gain = 0.000
Restored from archive | CPU: 0.000000 secs | Memory: 0.000000 MB |

Finished XDEF File Restore: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.039 . Memory (MB): peak = 3671.480 ; gain = 0.000
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 3671.480 ; gain = 0.000
INFO: [Project 1-111] Unisim Transformation Summary:
No Unisim elements were transformed.

b. Write a behavioral Verilog description of the design that will synthesize.

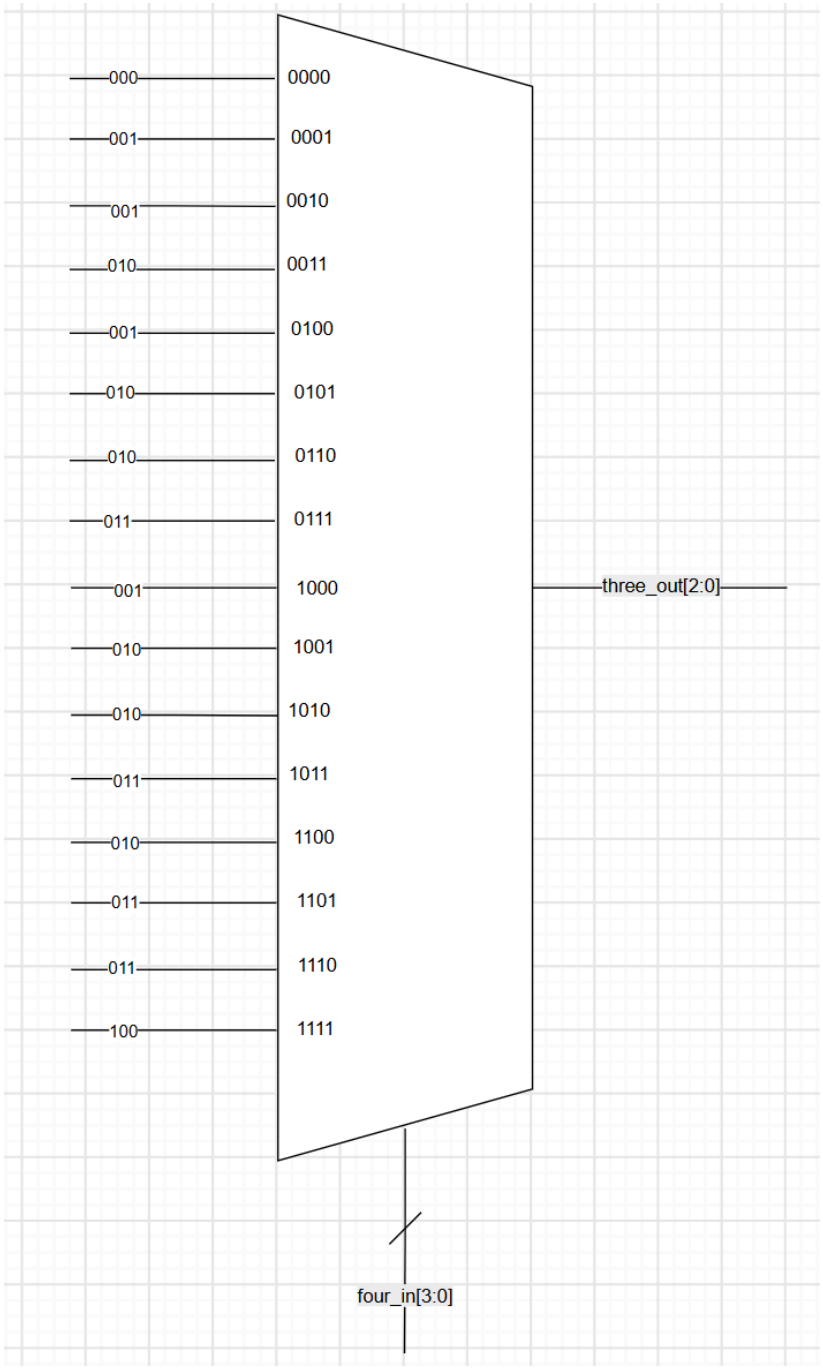
(used https://coddy.tech/playground/verilog to write and quick-test)
Module code and testbench:

```
module countones (
    input  [3:0] four_in,
    output reg [2:0] three_out
);

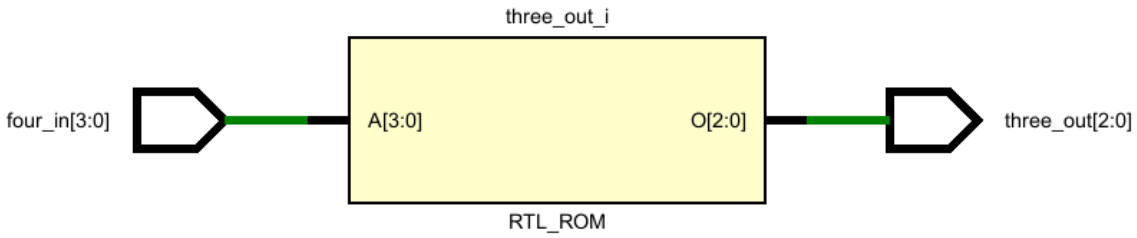
always @* begin
    case (four_in)
        4'b0001, 4'b0010, 4'b0100, 4'b1000: three_out = 3'b001;
        4'b1110, 4'b1101, 4'b1011, 4'b0111: three_out = 3'b011;
        4'b0000: three_out = 3'b000;
        4'b1111: three_out = 3'b100;
        default: three_out = 3'b010;
    endcase
end
endmodule

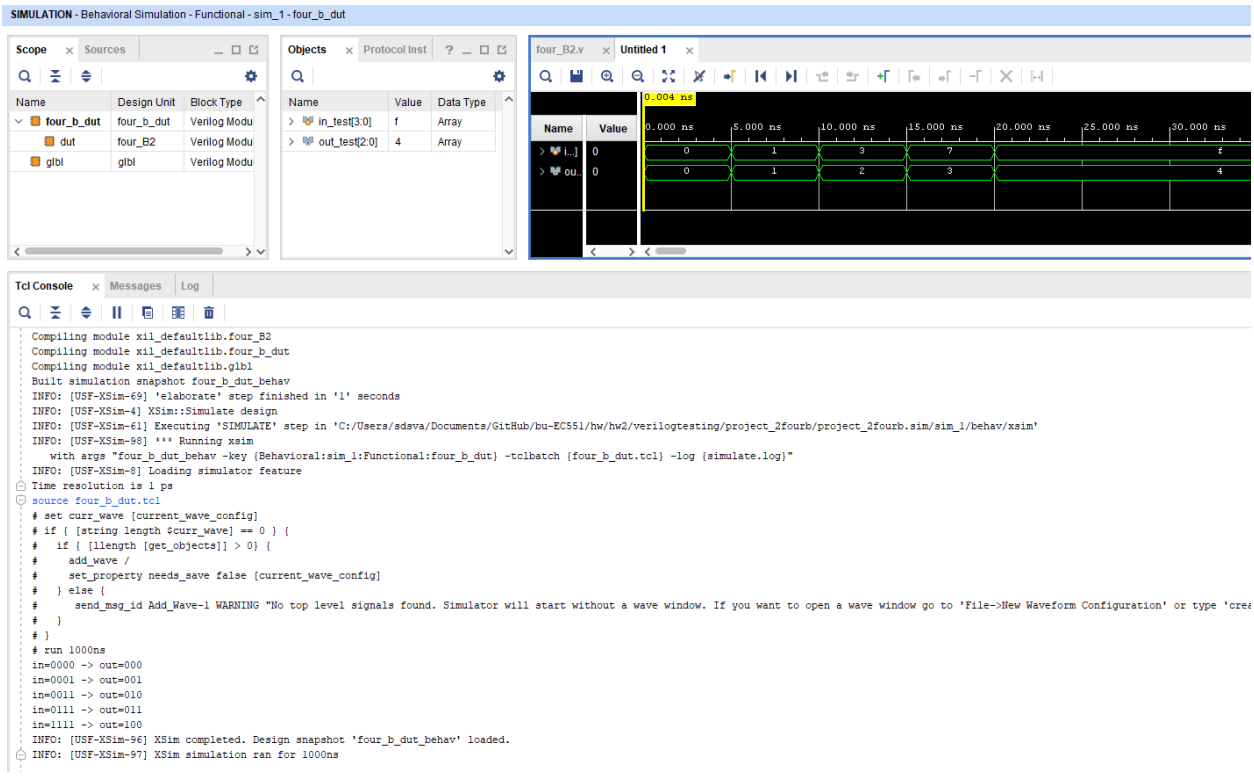
module test;
    reg  [3:0] in_test;
    wire [2:0] out_test;
    countones dut(in_test, out_test);
    initial begin
        in_test = 4'b0000; #5 $display("in=0000 -> out=%h%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0001; #5 $display("in=0001 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0011; #5 $display("in=0011 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0111; #5 $display("in=0111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b1111; #5 $display("in=1111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
    end
endmodule
```

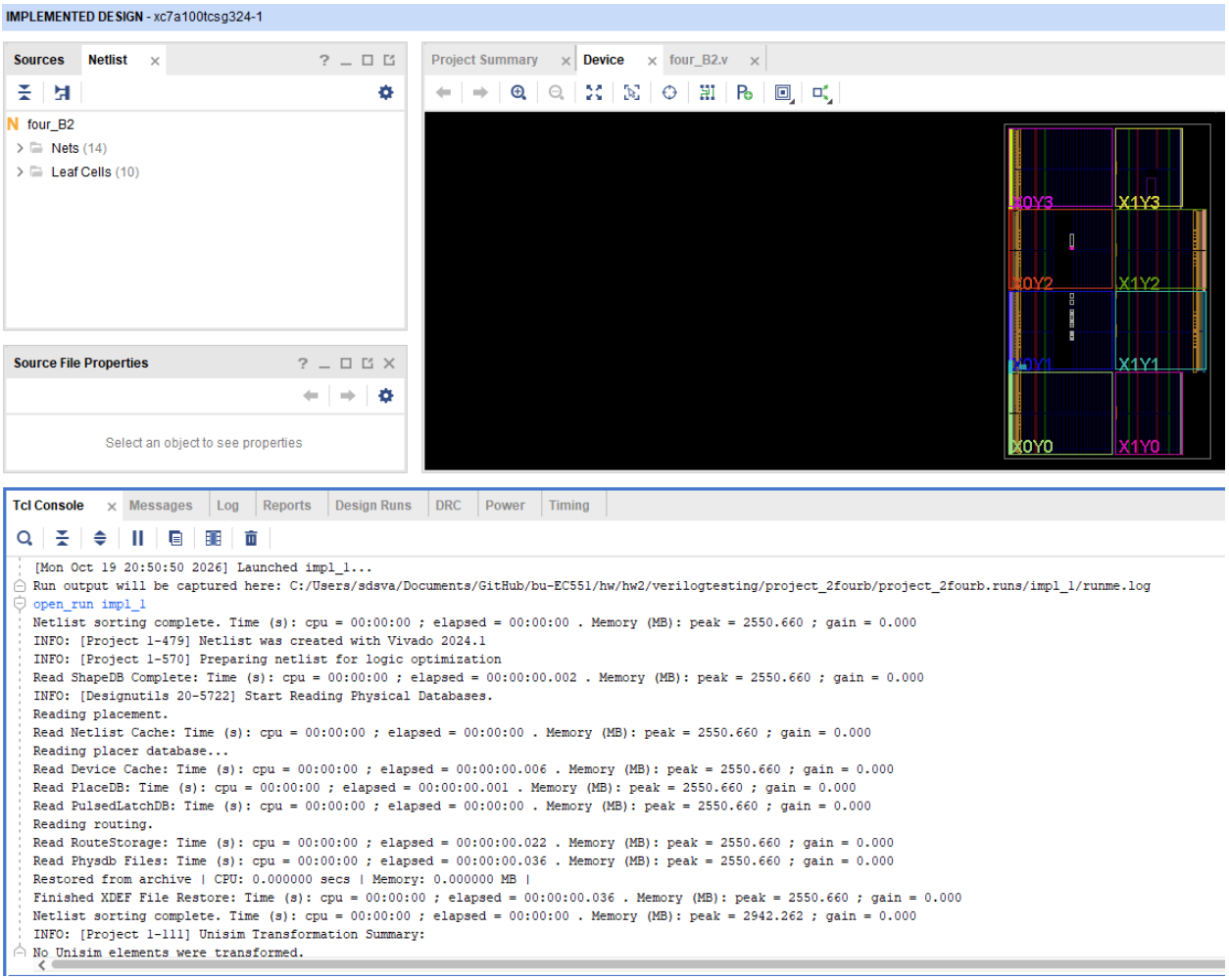
Below is the logic schematic, drawn on draw.io:


000
0000
001
0001
001
0010
010
0011
001
0100
010
0101
010
0110
011
0111
001
1000
010
1001
010
1010
011
1011
010
1100
011
1101
011
1110
100
1111
three_out[2:0]
four_in[3:0]

From Vivado:


three_out_i
four_in[3:0]
A[3:0]
O[2:0]
three_out[2:0]
RTL_ROM


SIMULATION - Behavioral Simulation - Functional - sim_1 - four_b_dut
Scope
Sources
Name
Design Unit
Block Type
four_b_dut
four_b_dut
Verilog Modu
dut
four_B2
Verilog Modu
glbl
glbl
Verilog Modu
Objects
Protocol Inst
Name
Value
Data Type
in_test[3:0]
f
Array
out_test[2:0]
4
Array
four_B2.v
Untitled 1
0.004 ns
Name
Value
0.000 ns
5.000 ns
10.000 ns
15.000 ns
20.000 ns
25.000 ns
30.000 ns
Tcl Console
Messages
Log
Compiling module xil_defaultlib.four_B2
Compiling module xil_defaultlib.four_b_dut
Compiling module xil_defaultlib.glbl
Built simulation snapshot four_b_dut_behav
INFO: [USF-XSim-69] 'elaborate' step finished in '1' seconds
INFO: [USF-XSim-4] XSim::Simulate design
INFO: [USF-XSim-61] Executing 'SIMULATE' step in 'C:/Users/sdsva/Documents/GitHub/bu-EC551/hw/hw2/verilogtesting/project_2fourb/project_2fourb.sim/sim_1/behav/xsim'
INFO: [USF-XSim-98] *** Running xsim
with args "four_b_dut_behav -key {Behavioral:sim_1:Functional:four_b_dut} -tclbatch {four_b_dut.tcl} -log {simulate.log}"
INFO: [USF-XSim-8] Loading simulator feature
Time resolution is 1 ps
source four_b_dut.tcl
set curr_wave [current_wave_config]
if { [string length $curr_wave] == 0 } {
if { [llength [get_objects]] > 0} {
add_wave /
set_property needs_save false [current_wave_config]
} else {
send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'cre
}
}
run 1000ns
in=0000 -> out=000
in=0001 -> out=001
in=0011 -> out=010
in=0111 -> out=011
in=1111 -> out=100
INFO: [USF-XSim-96] XSim completed. Design snapshot 'four_b_dut_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns

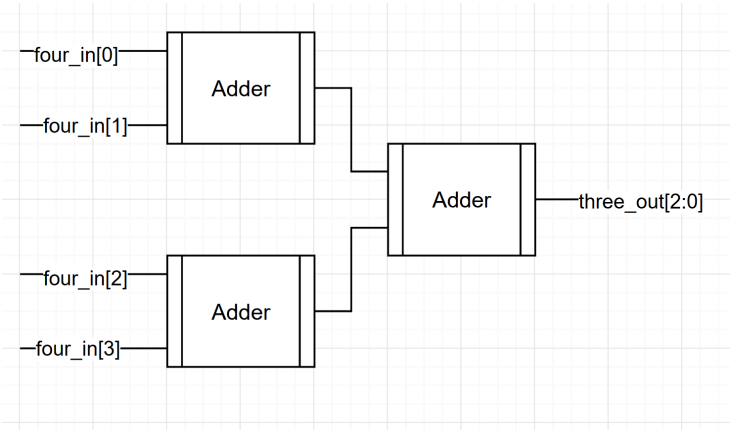

IMPLEMENTED DESIGN - xc7a100tcsg324-1
Sources
Netlist
four_B2
Nets (14)
Leaf Cells (10)
Source File Properties
Select an object to see properties
Project Summary
Device
four_B2.v
X0Y3
X1Y3
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0
Tcl Console
Messages
Log
Reports
Design Runs
DRC
Power
Timing
[Mon Oct 19 20:50:50 2026] Launched impl_1...
Run output will be captured here: C:/Users/sdsva/Documents/GitHub/bu-EC551/hw/hw2/verilogtesting/project_2fourb/project_2fourb.runs/impl_1/runme.log
open_run impl_1
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 2550.660 ; gain = 0.000
INFO: [Project 1-479] Netlist was created with Vivado 2024.1
INFO: [Project 1-570] Preparing netlist for logic optimization
Read ShapeDB Complete: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.002 . Memory (MB): peak = 2550.660 ; gain = 0.000
INFO: [Designutils 20-5722] Start Reading Physical Databases.
Reading placement.
Read Netlist Cache: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 2550.660 ; gain = 0.000
Reading placer database...
Read Device Cache: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.006 . Memory (MB): peak = 2550.660 ; gain = 0.000
Read PlaceDB: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.001 . Memory (MB): peak = 2550.660 ; gain = 0.000
Read PulsedLatchDB: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 2550.660 ; gain = 0.000
Reading routing.
Read RouteStorage: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.022 . Memory (MB): peak = 2550.660 ; gain = 0.000
Read Physdb Files: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.036 . Memory (MB): peak = 2550.660 ; gain = 0.000
Restored from archive | CPU: 0.000000 secs | Memory: 0.000000 MB |
Finished XDEF File Restore: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.036 . Memory (MB): peak = 2550.660 ; gain = 0.000
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 2942.262 ; gain = 0.000
INFO: [Project 1-111] Unisim Transformation Summary:
No Unisim elements were transformed.

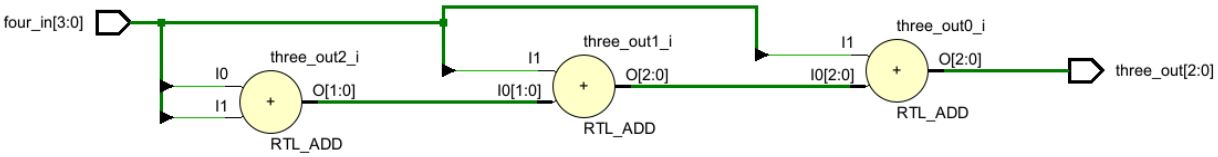
c. Write a behavioral Verilog description of the design that synthesizes AND the resultant circuit is different than part b.

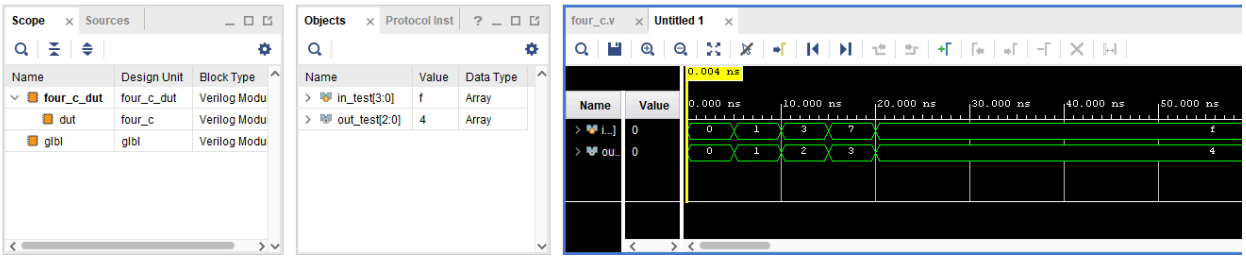
```
module countones (
    input  [3:0] four_in,
    output reg [2:0] three_out
);
    always @(*) begin
        three_out = four_in[0] + four_in[1] + four_in[2] +
four_in[3];
    end
endmodule

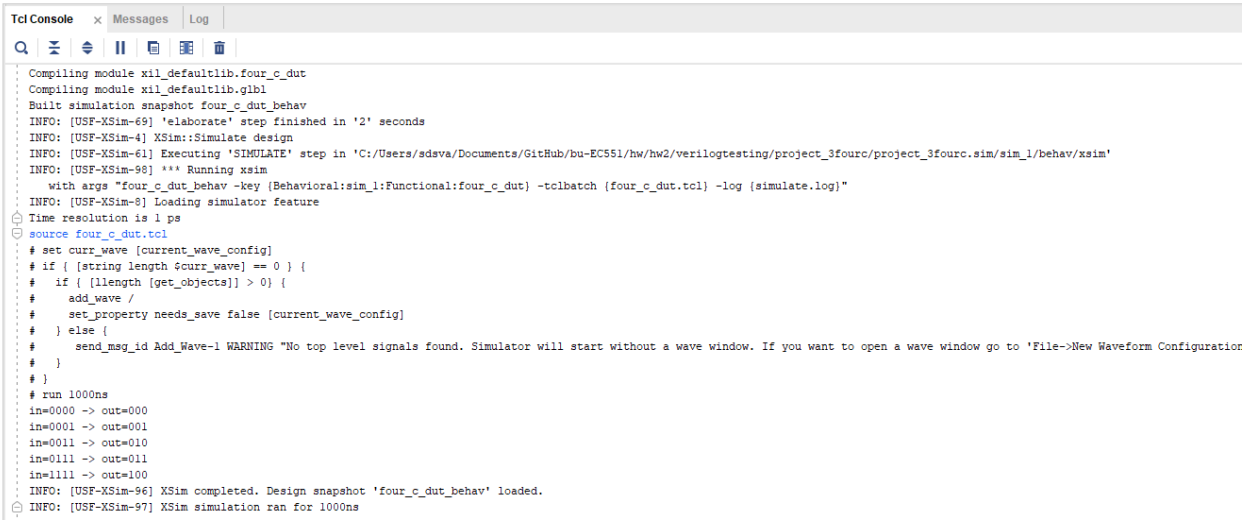

module test;
    reg  [3:0] in_test;
    wire [2:0] out_test;
    countones dut(in_test, out_test);
    initial begin
        in_test = 4'b0000; #5 $display("in=0000 -> out=%h%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0001; #5 $display("in=0001 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0011; #5 $display("in=0011 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b0111; #5 $display("in=0111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
        in_test = 4'b1111; #5 $display("in=1111 -> out=%h",
out_test[2],out_test[1], out_test[0]);
    end
endmodule
```

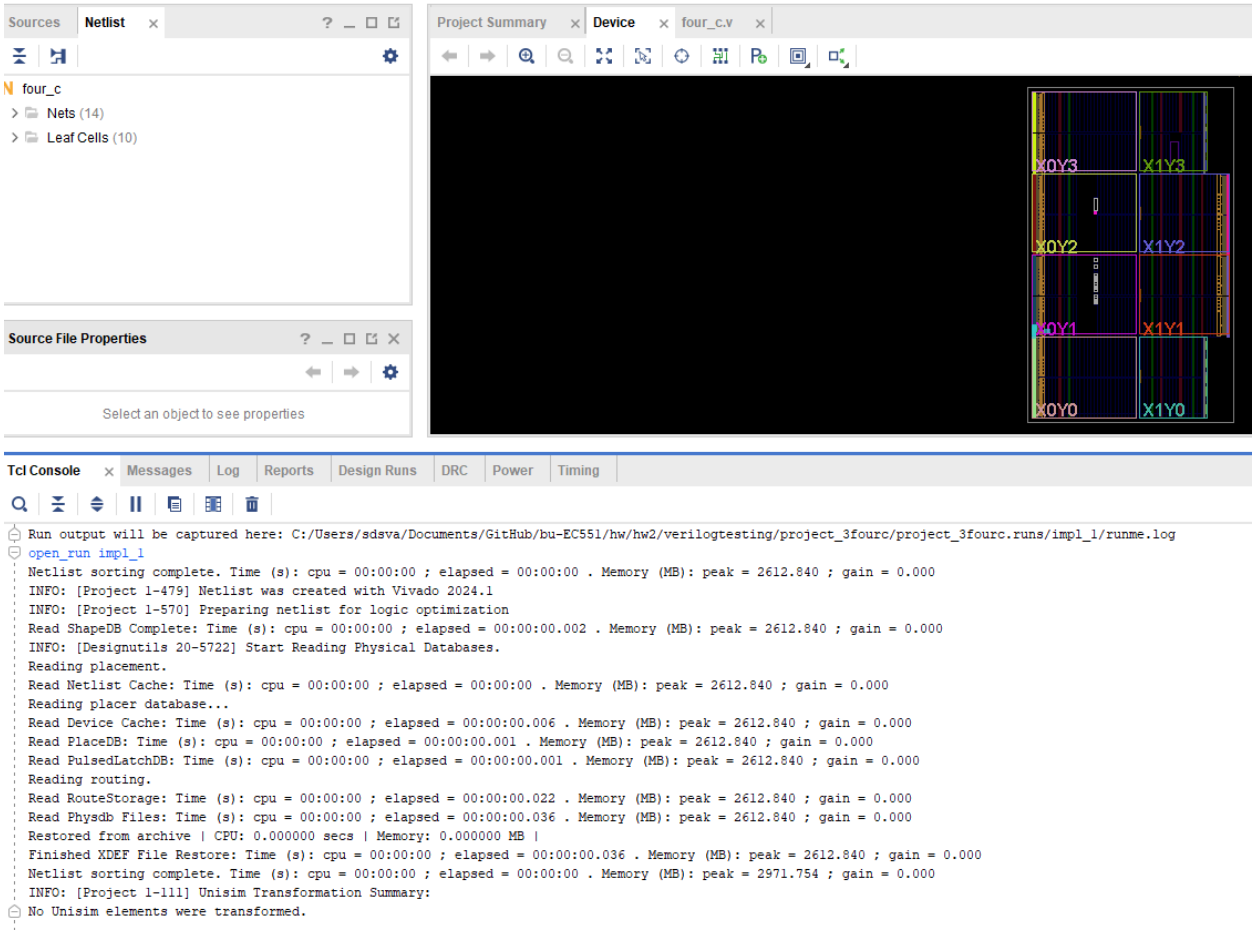
Logic schematic below, drawn with draw.io:


four_in[0]
four_in[1]
four_in[2]
four_in[3]
Adder
Adder
Adder
three_out[2:0]


four_in[3:0]
three_out2_i
I0
I1
O[1:0]
RTL_ADD
three_out1_i
I1
I0[1:0]
O[2:0]
RTL_ADD
three_out0_i
I1
I0[2:0]
O[2:0]
RTL_ADD
three_out[2:0]


Scope
Sources
Name
Design Unit
Block Type
four_c_dut
four_c_dut
Verilog Modu
dut
four_c
Verilog Modu
glbl
glbl
Verilog Modu
Objects
Protocol Inst
Name
Value
Data Type
in_test[3:0]
f
Array
out_test[2:0]
4
Array
four_c.v
Untitled 1
0.004 ns
Name
Value
0.000 ns
10.000 ns
20.000 ns
30.000 ns
40.000 ns
50.000 ns


Tcl Console
Messages
Log
Compiling module xil_defaultlib.four_c_dut
Compiling module xil_defaultlib.glbl
Built simulation snapshot four_c_dut_behav
INFO: [USF-XSim-69] 'elaborate' step finished in '2' seconds
INFO: [USF-XSim-4] XSim::Simulate design
INFO: [USF-XSim-61] Executing 'SIMULATE' step in 'C:/Users/sdsva/Documents/GitHub/bu-EC551/hw/hw2/verilogtesting/project_3fourc/project_3fourc.sim/sim_1/behav/xsim'
INFO: [USF-XSim-98] *** Running xsim
with args "four_c_dut_behav -key {Behavioral:sim_1:Functional:four_c_dut} -tclbatch {four_c_dut.tcl} -log {simulate.log}"
INFO: [USF-XSim-8] Loading simulator feature
Time resolution is 1 ps
source four_c_dut.tcl
set curr_wave [current_wave_config]
if { [string length $curr_wave] == 0 } {
if { [llength [get_objects]] > 0} {
add_wave /
set_property needs_save false [current_wave_config]
} else {
send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration
}
}
run 1000ns
in=0000 -> out=000
in=0001 -> out=001
in=0011 -> out=010
in=0111 -> out=011
in=1111 -> out=100
INFO: [USF-XSim-96] XSim completed. Design snapshot 'four_c_dut_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns

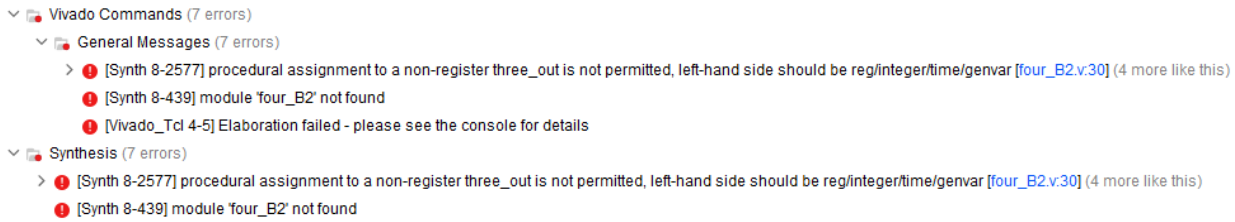


d. Write a Verilog description that WILL NOT synthesize. Show the error messages/result that indicates that it will not synthesize. Explain why these occur.

```
module four_d(
    input [3:0] four_in,
    output reg [2:0] three_out
    );

always @* begin
    case (four_in)
        4'b0001, 4'b0010, 4'b0100, 4'b1000: three_out = 3'b001;
        4'b1110, 4'b1101, 4'b1011, 4'b0111: three_out = 3'b011;
        4'b0000: three_out = 3'b000;
        4'b1111: three_out = 3'b100;
        default: three_out = 3'b010;
    endcase
end

endmodule
```



5. Consider a 4-bit counter. This will be a sequential logic circuit (i.e. FSM). The circuit should have a reset signal. This should set it to the initial output state of 4’b0000. The only other input should be a clock signal. Each clock tick should advance the counter. The counter should reset after 4’b1111.

Provide verilog code, schematics, and relevant synthesis messages for all four parts.

a. Write a structural Verilog description of the design that synthesizes.

To be honest I’m unsure how to do this

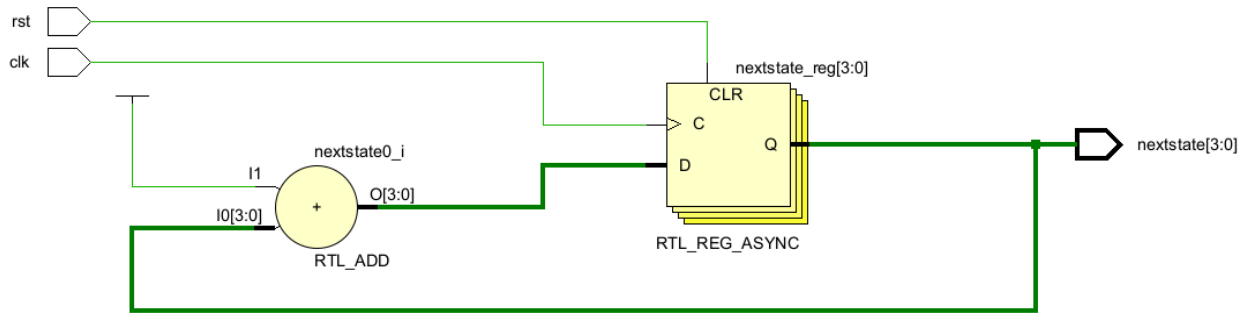
b. Write a behavioral Verilog description of the design that will synthesize.

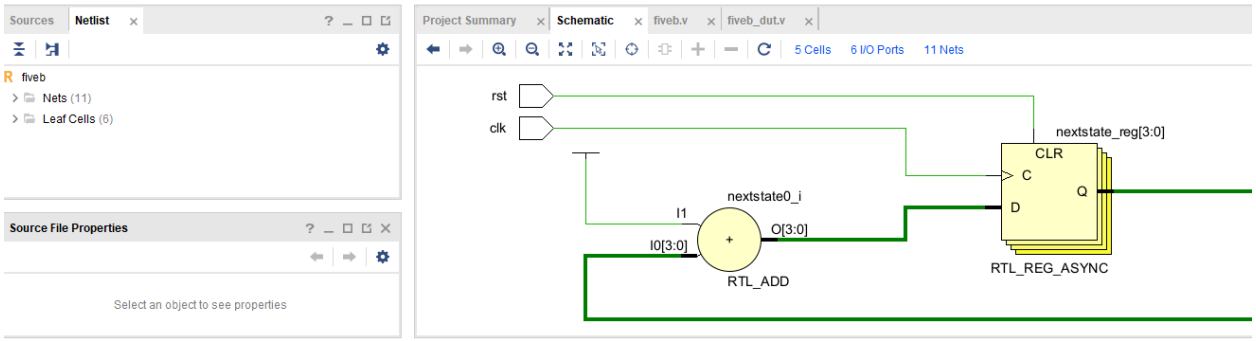
```
module countfifteen (
    input clk, input rst,
    output reg [3:0] nextstate
);
always @(posedge clk, posedge rst) begin
    if (rst) begin
        nextstate <= 4'b0000;
    end else begin
        nextstate <= nextstate + 4'b0001;
    end
end
endmodule

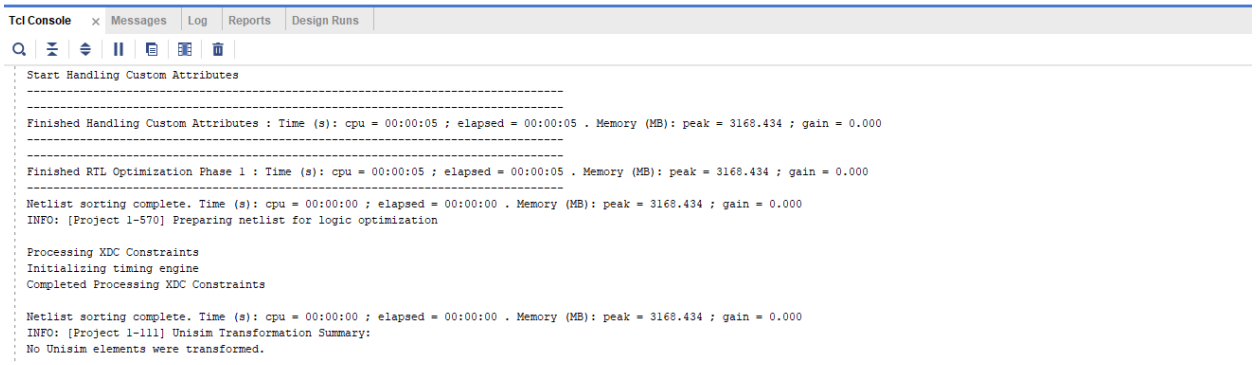

module test;
    reg  clk = 0;
    reg rst = 0;
    wire [3:0] out_next;
    countfifteen dut(clk, rst, out_next);
    initial clk = 0;
    always #5 clk = ~clk;

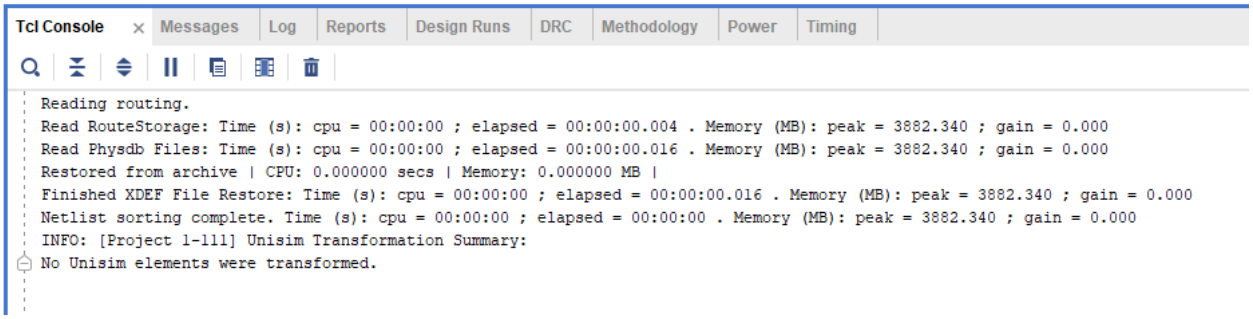

    initial begin
        rst = 1;#10
```

```
        rst = 0;
        $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        repeat(20) begin
            #10 $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        end
        rst = 1'b1; $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]); #10
        repeat(10) begin
            $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        end
        rst = 0;
        $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        repeat(25) begin
            #10 $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        end
        #300 $finish;
    end
endmodule
```









c. Write a behavioral Verilog description of the design that synthesizes AND the resultant circuit is different than part b.

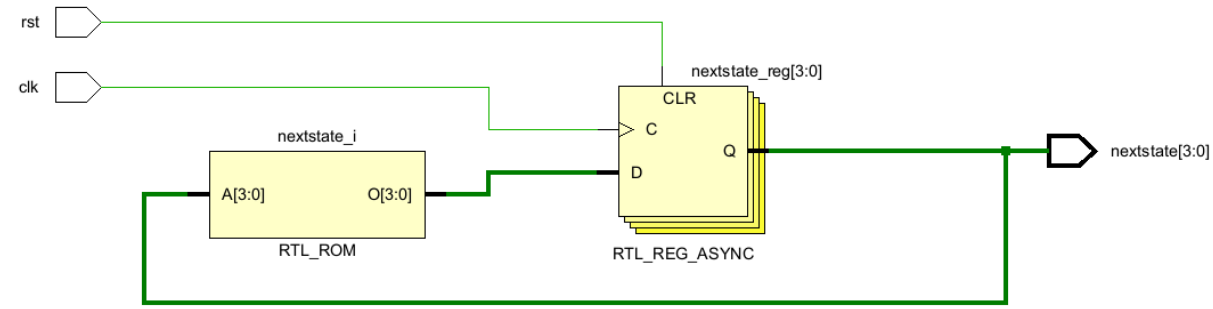
```
module countfifteen (
    input clk, input rst,
    output reg [3:0] nextstate
);
always @(posedge clk, posedge rst) begin
    if (rst) begin
        nextstate <= 4'b0000;
    end else begin
        case (nextstate)
            4'b0000: nextstate <= 4'b0001;
```

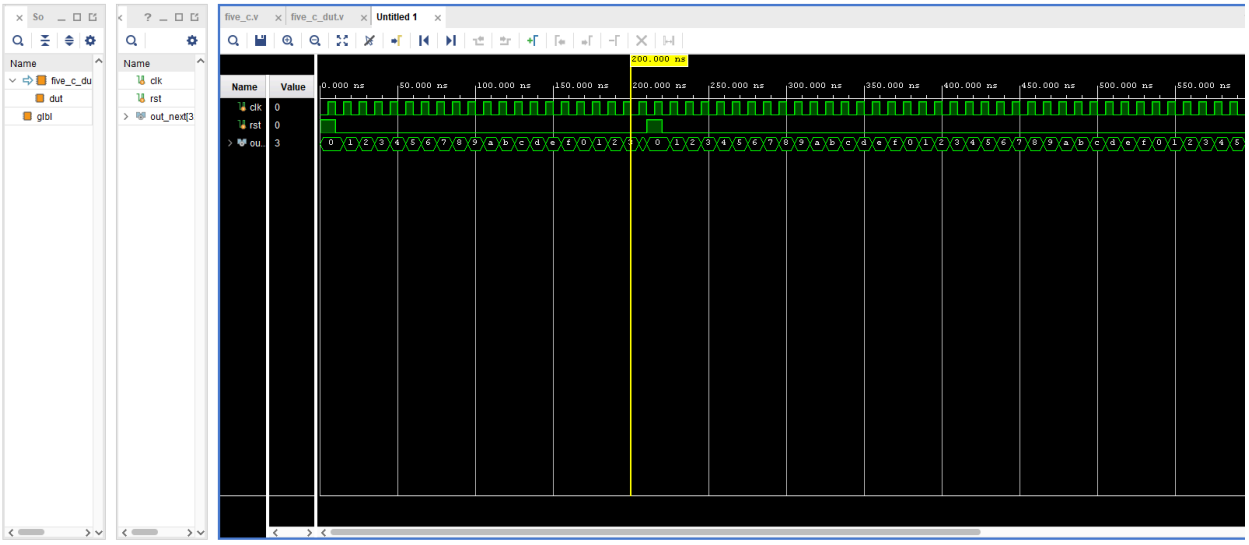
```
            4'b0001: nextstate <= 4'b0010;
            4'b0010: nextstate <= 4'b0011;
            4'b0011: nextstate <= 4'b0100;
            4'b0100: nextstate <= 4'b0101;
            4'b0101: nextstate <= 4'b0110;
            4'b0110: nextstate <= 4'b0111;
            4'b0111: nextstate <= 4'b1000;
            4'b1000: nextstate <= 4'b1001;
            4'b1001: nextstate <= 4'b1010;
            4'b1010: nextstate <= 4'b1011;
            4'b1011: nextstate <= 4'b1100;
            4'b1100: nextstate <= 4'b1101;
            4'b1101: nextstate <= 4'b1110;
            4'b1110: nextstate <= 4'b1111;
            4'b1111: nextstate <= 4'b0000;
        endcase
    end
end
endmodule

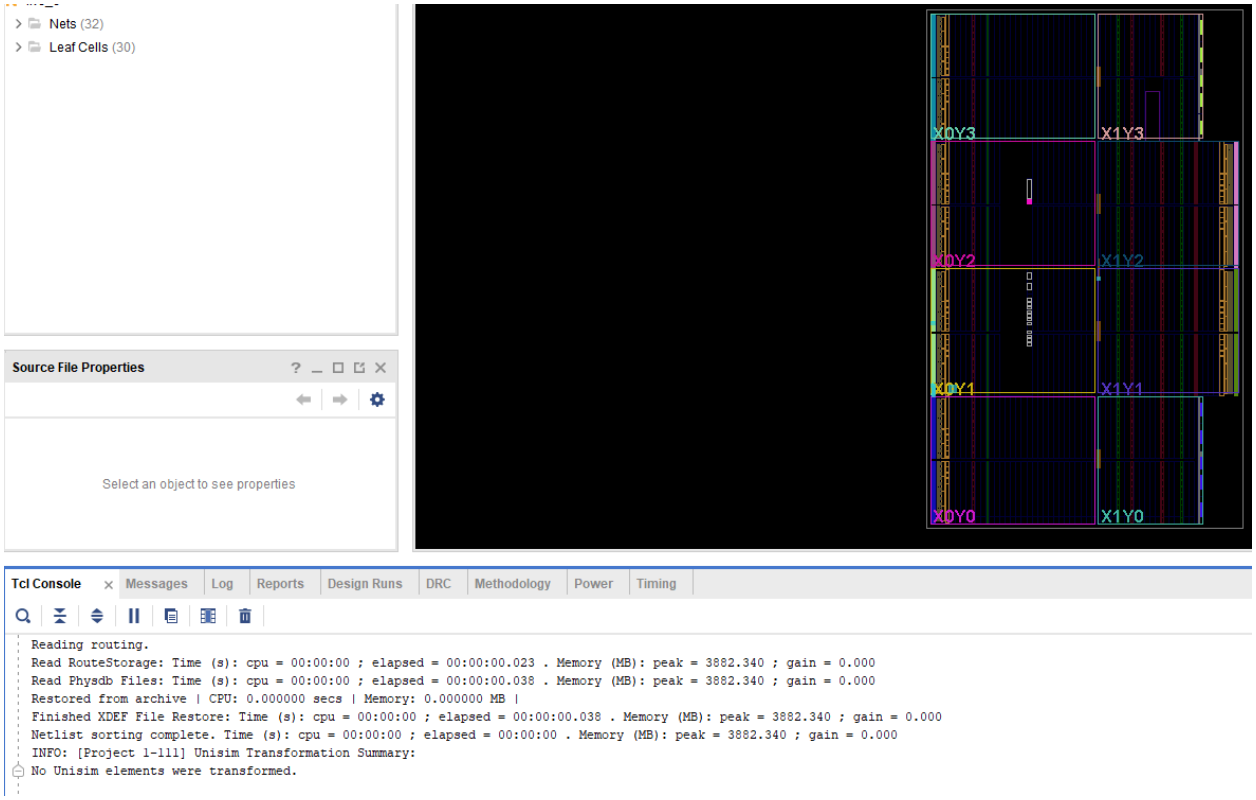

module test;
    reg  clk = 0;
    reg rst = 0;
    wire [3:0] out_next;
    countfifteen dut(clk, rst, out_next);
    initial clk = 0;
    always #5 clk = ~clk;


    initial begin
        rst = 1;#10
        rst = 0;
        $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        repeat(20) begin
            #10 $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        end
        rst = 1'b1; $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]); #10
        repeat(10) begin
            $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        end
```

```
        rst = 0;
        $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        repeat(25) begin
            #10 $display("in=0000 -> out=%h%h",out_next[3],
out_next[2],out_next[1], out_next[0]);
        end
        #300 $finish;
    end
endmodule
```



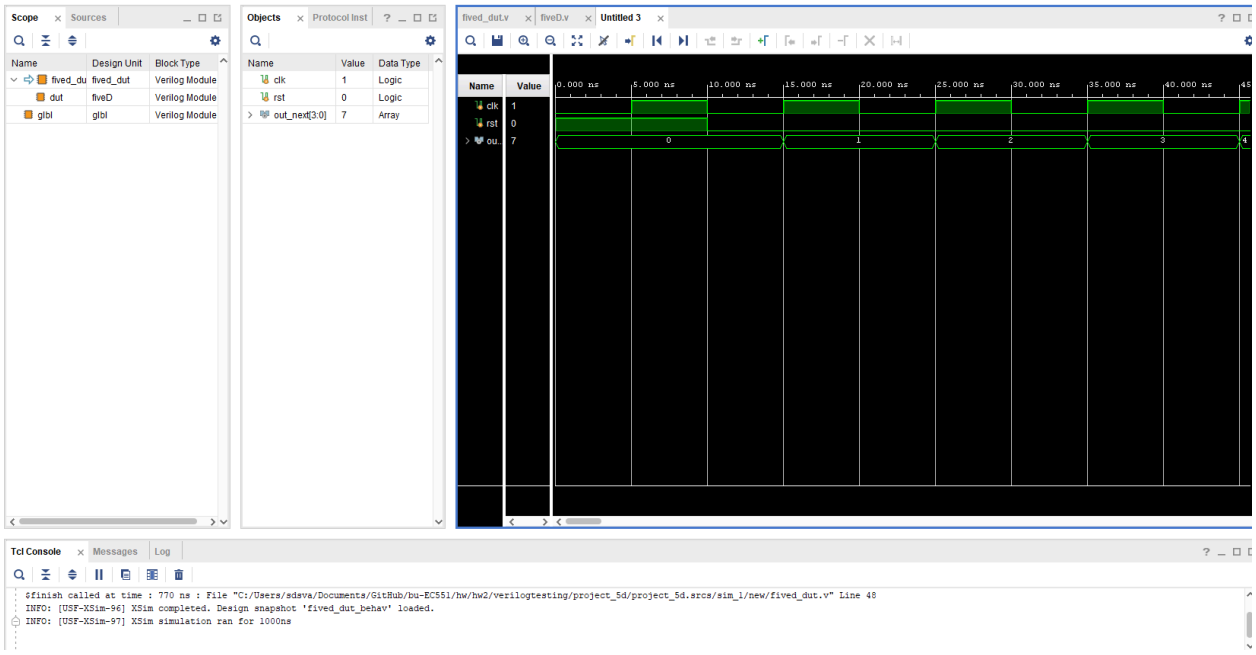




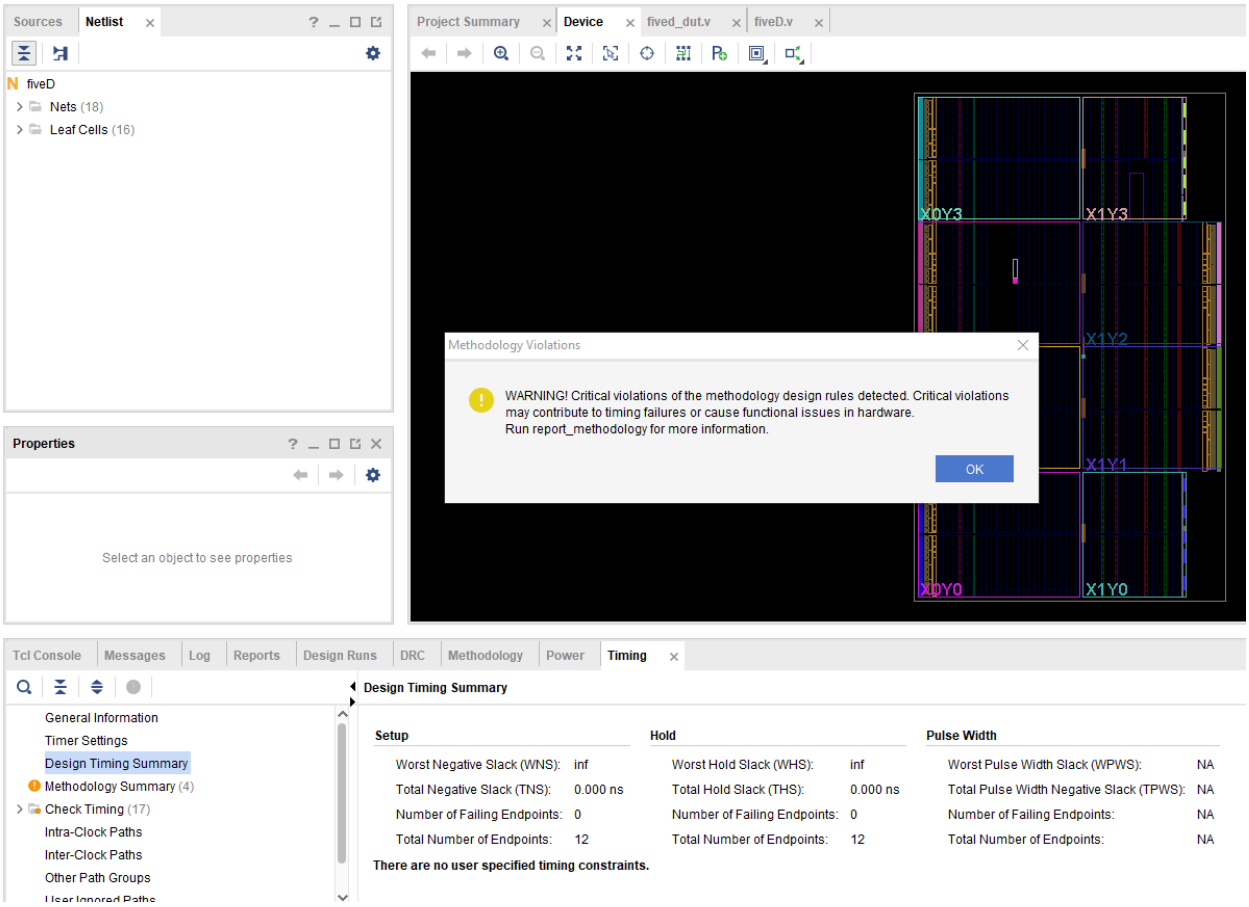
d. Write a Verilog description that WILL NOT synthesize. Show the error messages/result that indicates that it will not synthesize. Explain why these occur.

```
module countfifteen (
    input clk, input rst,
    output reg [3:0] nextstate
);
always @(posedge clk, posedge rst) begin
    if (rst) begin
        nextstate = 4'b0000;
    end else begin
        case (nextstate)
            4'b0000: nextstate <= 4'b0001;
            4'b0001: nextstate <= 4'b0010;
            4'b0010: nextstate <= 4'b0011;
            4'b0011: nextstate <= 4'b0100;
            4'b0100: nextstate <= 4'b0101;
            4'b0101: nextstate <= 4'b0110;
            4'b0110: nextstate <= 4'b0111;
            4'b0111: nextstate <= 4'b1000;
            4'b1000: nextstate <= 4'b1001;
            4'b1001: nextstate <= 4'b1010;
            4'b1010: nextstate <= 4'b1011;
```

```
            4'b1011: nextstate <= 4'b1100;
            4'b1100: nextstate <= 4'b1101;
            4'b1101: nextstate <= 4'b1110;
            4'b1110: nextstate <= 4'b1111;
            4'b1111: nextstate <= 4'b0000;
        endcase
    end
```



I mixed up blocking/nonblocking statements, to get:


Sources
Netlist
fiveD
Nets (18)
Leaf Cells (16)
Properties
Select an object to see properties
Project Summary
Device
fived_dut.v
fiveD.v
X0Y3
X1Y3
X1Y2
X1Y1
X0Y0
X1Y0
Methodology Violations
WARNING! Critical violations of the methodology design rules detected. Critical violations may contribute to timing failures or cause functional issues in hardware.
Run report_methodology for more information.
OK
Tcl Console
Messages
Log
Reports
Design Runs
DRC
Methodology
Power
Timing
General Information
Timer Settings
Design Timing Summary
Methodology Summary (4)
Check Timing (17)
Intra-Clock Paths
Inter-Clock Paths
Other Path Groups
Design Timing Summary
Setup
Worst Negative Slack (WNS): inf
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 12
Hold
Worst Hold Slack (WHS): inf
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 12
Pulse Width
Worst Pulse Width Slack (WPWS): NA
Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: NA
Total Number of Endpoints: NA
There are no user specified timing constraints.